BASH SCRIPTING FUNDAMENTALS

Files, Pipes & Redirection

Streams, pipelines, and gluing Unix tools together

Merit Advisory
Bash Scripting Fundamentals Bootcamp
Beginner-friendly lecture materials
July 2026

WHAT WE WILL COVER

Agenda

1. **Section 1: Stream Fundamentals** — 4 slides
2. **Section 2: Output Redirection** — 4 slides
3. **Section 3: stderr and Silence** — 4 slides
4. **Section 4: Pipes and Pipelines** — 4 slides
5. **Section 5: tee and Logging** — 4 slides
6. **Section 6: Here Input and Substitution** — 4 slides
7. **Section 7: File Descriptors and exec** — 4 slides
8. **Section 8: Temporary Files and Cleanup** — 4 slides

9 **Section 9: Batching and File Streams** — 4 slides

10 **Section 10: Filesystem Helpers** — 4 slides

11 **Section 11: Disk Awareness and Checkpoint** — 4 slides

1 Section 1

Stream Fundamentals

1.1 stdout carries normal output · stdin supplies input

- **stdout** is file descriptor 1.
- Commands write useful results to **stdout** by default.
- Pipelines connect **stdout** from one command to stdin of the next.
- **stdin** is file descriptor 0.
- Many filters read **stdin** when no file is named.
- A pipe or input redirect can feed **stdin**.

Examples

```
# Print normal output
printf 'ready\n'
# Redirect stdout
printf 'ready\n' > status.txt
# Read from a file
wc -l < names.txt
# Read from a pipe
printf 'a\nb\n' | wc -l
```

1.2 stderr carries diagnostics · File descriptors identify streams

- **stderr** is file descriptor 2.
- Errors and warnings should not mix with normal data.
- Keeping **stderr** separate makes pipelines safer.
- A **file** descriptor is a small number attached to an open stream.
- Bash starts every command with descriptors 0, 1, and 2.
- Redirection changes where those descriptors point.

Examples

```
# Show an error stream
ls missing.txt
# Capture only errors
ls missing.txt 2> errors.log
# Name the standard fds
printf 'stdin=0 stdout=1 stderr=2\n'
# Send fd 1 to a file
date 1> now.txt
```

1.3 Streams are not always files · Input redirection feeds stdin

- A stream may point at a terminal, pipe, socket, or file.
- **The** command usually does not need to know the target type.
- This abstraction is why small Unix tools compose well.
- The < operator opens a file for fd 0.
- **The** command reads the file as standard input.
- This avoids passing a filename when a tool expects stdin.

Examples

```
# Terminal target
printf 'hello\n'
# File target
printf 'hello\n' > hello.txt
# Count file lines
wc -l < app.log
# Sort redirected input
sort < names.txt
```

1.4 Output redirection captures stdout · Exit status is separate from streams

- The > and >> operators target fd 1 unless another fd is named.
- **Output** redirection happens before the command body runs.
- The shell opens the target file, not the command.
- A command can print text and still fail.
- A command can be quiet and still succeed.
- Check exit status separately from captured **output**.

Examples

```
# Capture a listing
ls > listing.txt
# Append a listing
ls >> listing.txt
# Quiet success
true > out.txt
# Failed command with stderr
grep needle missing.txt 2> err.txt
```

2 Section 2

Output Redirection

2.1 Overwrite with > · Append with >>

- > creates the target file if needed.
- > truncates an existing target before writing.
- Use it when the old content should be replaced.
- >> creates the target file if needed.
- >> keeps existing content and writes at the end.
- **Append** is the usual choice for ongoing logs.

Examples

```
# Create a report
date > report.txt
# Replace the report
hostname > report.txt
# Append one line
date >> run.log
# Append command output
uptime >> run.log
```

2.2 Clobbering is easy · noclobber protects overwrites

- A mistaken > can erase a file before the command starts.
- Review important output paths before pressing Enter.
- Use backups or temporary files for valuable data.
- set -o **noclobber** makes > refuse existing files.
- >| intentionally overrides **noclobber** for one redirect.
- **This** is a shell safety option, not a file permission.

Examples

```
# Dangerous overwrite
printf 'new\n' > notes.txt
# Safer append
printf 'new\n' >> notes.txt
# Enable protection
set -o noclobber
# Force one overwrite
printf 'ok\n' >| output.txt
```

2.3 Redirect a command group · Create or truncate with :

- Braces group several commands in the current shell.
- One **redirect** can capture the whole group.
- Group redirects keep related output together.
- : is a do-nothing command that succeeds.
- Redirecting : can create an empty file.
- **This** is explicit when you intend to truncate.

Examples

```
# Write a small report
{ date; hostname; } > report.txt
# Append a grouped log
{ echo start; date; } >> run.log
# Create empty file
: > empty.txt
# Reset a log
: > run.log
```

2.4 sudo and redirects are separate · Redirect to predictable paths

- **Redirection** is performed by your current shell.
- sudo before a command does not sudo the **redirect**.
- Use tee when root must write the destination.
- Relative **redirect** paths depend on the current directory.
- Variables make output destinations easier to audit.
- Quote paths stored in variables.

Examples

```
# Often fails
sudo echo ok > /etc/example.conf
# Use tee
echo ok | sudo tee /etc/example.conf
# Use a variable
log='logs/run.log'
date >> "$log"
out="$PWD/output.txt"
printf 'ok\n' > "$out"
```

3 Section 3

stderr and Silence

3.1 Redirect stderr with 2> · Separate stdout and stderr

- 2> changes file descriptor 2 only.
- stdout still goes wherever it was already going.
- Use **separate** error logs when data must stay clean.
- You can send stdout and stderr to different files.
- This keeps results **separate** from diagnostics.
- **Separate** logs make automated parsing safer.

Examples

```
# Capture errors
grep root missing.txt 2> errors.log
# Keep stdout visible
find /root -maxdepth 1 2> denied.log
# Two files
cmd > output.log 2> errors.log
# Find example
find . -name '*.sh' > files.txt 2> find.err
```

3.2 Merge stderr into stdout with 2>&1 · Redirection order changes results

- 2>&1 makes fd 2 point to the current fd 1 target.
- Order matters because redirects are processed left to right.
- Use this when one combined log is desired.
- cmd >file 2>&1 sends both streams to file.
- cmd 2>&1 >file sends stderr to the old stdout.
- Read redirect chains from left to right.

Examples

```
# Combined file
cmd > combined.log 2>&1
# Combined pipe
cmd 2>&1 | tee combined.log
# Both to file
cmd > all.log 2>&1
# stderr stays visible
cmd 2>&1 > out.log
```

3.3 Use &> for both streams · Discard stdout with >/dev/null

- &> file redirects stdout and stderr together in Bash.
- **It** is shorter than > file 2>&1.
- **It** is Bash-specific, not portable sh syntax.
- /dev/null accepts data and throws it away.
- >/dev/null silences normal output only.
- Errors still appear unless stderr is redirected too.

Examples

```
# Combined overwrite
cmd &> all.log
# Combined append
cmd &>> all.log
# Ignore normal output
curl -s https://example.com >/dev/null
# Keep errors visible
find . -name '*.tmp' >/dev/null
```

3.4 Discard stderr with 2>/dev/null · Silence both streams

- 2>/dev/null silences diagnostics only.
- stdout remains available for data.
- Use it carefully because errors can explain bad results.
- Redirect stdout to /dev/null first.
- Then point stderr at the same place.
- Use quiet commands only when failures are handled elsewhere.

Examples

```
# Hide permission errors
find / -name passwd 2>/dev/null
# Keep matches
grep root /etc/* 2>/dev/null
# Portable Bash pattern
cmd >/dev/null 2>&1
# Bash shortcut
cmd &>/dev/null
```

4 Section 4

Pipes and Pipelines

4.1 Pipe with | · Pipelines read left to right

- | connects stdout from the left command to stdin of the **right** command.
- Pipes avoid temporary files for simple transformations.
- Only stdout flows through a **pipe** by default.
- Each stage receives the previous stage's stdout.
- Think of a pipeline as data moving through filters.
- Place broad producers before narrow filters.

Examples

```
# Count entries
ls | wc -l
# Filter output
ps aux | grep ssh
# Filter then count
printf 'a\nb\n' | grep a | wc -l
# List then sort
ls | sort
```

4.2 Filters transform streams · Pipe to commands, redirect to files

- A filter reads stdin and writes stdout.
- Classic **filters** include sort, cut, grep, sed, and awk.
- **Filters** are easiest to test one stage at a time.
- Use | when another command should read the data.
- Use > or >> when a file should store the data.
- Mix them when a pipeline result should be saved.

Examples

```
# Sort a stream
printf 'b\na\n' | sort
# Take a column
printf 'a:b\n' | cut -d: -f1
# Pipe to a command
ls | wc -l
# Save pipeline output
ls | sort > files.txt
```

4.3 Pipeline status normally uses the last command · pipefail previews safer pipelines

- Bash returns the status of the last **pipeline** command by default.
- An early failure can be hidden by a later success.
- This matters in scripts that depend on reliable failure checks.
- set -o pipefail makes a **pipeline** fail when any stage fails.
- **It** is commonly paired with strict script settings.
- Use it deliberately because it changes failure behavior.

Examples

```
# Last command wins
false | true
echo $?
# Fail at the end
true | false
echo $?
# Enable pipefail
set -o pipefail
set -o pipefail
false | true
```

4.4 Buffering can delay output · yes generates repeated input

- Programs may buffer output differently in pipes than terminals.
- Buffered output can make live pipelines appear stuck.
- Prefer tools with line-buffer options for live logs.
- **yes** prints a string repeatedly until stopped.
- It can feed prompts in controlled examples.
- Be careful because it produces unlimited output.

Examples

```
# Live log pipeline
tail -f app.log | grep ERROR
# Line-buffer grep
tail -f app.log | grep --line-buffered ERROR
# Generate y
yes | head -n 3
# Generate a word
yes ok | head -n 2
```

5 Section 5

tee and Logging

5.1 tee writes to screen and file · tee -a appends

- **tee** copies stdin to stdout and files.
- It lets you watch output while saving it.
- Without -a, **tee** overwrites the target file.
- **tee** -a keeps existing file content.
- Append mode is useful for long-running logs.
- It mirrors the difference between > and >>.

Examples

```
# Save and display
make 2>&1 | tee build.log
# Write one line
echo ready | tee status.txt
# Append output
date | tee -a run.log
# Append errors too
cmd 2>&1 | tee -a all.log
```

5.2 Log and screen together · Append timestamps to logs

- A combined **log** captures what the operator saw.
- 2>&1 before tee includes errors in the **log**.
- The pipeline still displays the same stream.
- A timestamp gives each **log** entry context.
- **Use** command substitution to generate the time.
- Keep the timestamp format sortable.

Examples

```
# Capture a run
./backup.sh 2>&1 | tee backup.log
# Append a run
./backup.sh 2>&1 | tee -a backup.log
# Date prefix
printf '%s start\n' "$(date +%F_%T)" >> run.log
# Group with timestamp
{ date '+%F %T'; echo done; } >> run.log
```

5.3 Use a simple logging function · Split output and error logs

- A function keeps log formatting consistent.
- Append inside the function to avoid repeated redirects.
- Quote the message so spaces stay together.
- Separate logs keep successful data apart from diagnostics.
- **This** is useful when stdout is machine-readable.
- stderr can be inspected without parsing results.

Examples

```
# Define logger
log(){ printf '%s %s\n' "$(date +%F_%T)" "$*" >> app.log; }
# Call logger
log 'backup started'
# Separate logs
./job.sh > job.out 2> job.err
# Append separately
./job.sh >> job.out 2>> job.err
```

5.4 Rotate a simple log · tee affects pipeline status

- Rotation moves an old log before starting a new one.
- A timestamped suffix keeps previous runs available.
- Large production logs need a real rotation tool.
- A command piped to tee becomes part of a pipeline.
- Without pipefail, the pipeline status can hide the first command.
- Enable pipefail when **tee** logs important failures.

Examples

```
# Move old log
mv app.log "app.$(date +%F_%H%M%S).log"
# Start fresh
: > app.log
# Hidden failure risk
false | tee run.log
echo $?
set -o pipefail
false | tee run.log
```

6 Section 6

Here Input and Substitution

6.1 Here-documents use <<EOF · Quoted here-doc delimiters disable expansion

- A here-document feeds multiple lines to stdin.
- The delimiter marks where the input ends.
- Unquoted delimiters allow normal shell expansion.
- <<'EOF' treats the body literally.
- Variables and command substitutions are not expanded.
- Use it for templates containing shell syntax.

Examples

```
# Feed cat
cat <<EOF
hello
# ...
# Create a file
cat > note.txt <<EOF
hello
# ...
cat <<'EOF'
$HOME
cat <<'EOF'
$(date)
```

6.2 Tab-stripping here-documents use <<-EOF · Here-strings use <<<

- <<- removes leading tab characters from body lines.
- Only real tabs are stripped, not spaces.
- This helps indent here-docs inside scripts.
- A here-string feeds one string to stdin.
- Bash adds a trailing newline.
- **It** is handy for simple filters and tests.

Examples

```
# Indented body
cat <<-EOF
indented
# ...
# Indented template
cat > file.txt <<-EOF
value
# ...
grep root <<< "$USER"
wc -w <<< "one two"
```

6.3 Process substitution provides input paths · Process substitution can capture output

- <(command) exposes command output as a temporary path.
- It lets file-oriented tools read generated streams.
- **The** command runs asynchronously while the consumer reads.
- >(command) exposes a writable path connected to a command.
- Data written to that path becomes stdin for the command.
- Use it when a program can write only to filenames.

Examples

```
# Compare streams
diff <(sort old.txt) <(sort new.txt)
# Loop over generated data
while read -r x; do echo "$x"; done < <(printf 'a\n')
# Split stdout
cmd > >(tee out.log)
# Compress by path
tar cf >(gzip > files.tar.gz) dir
```

6.4 /dev/stdin, /dev/stdout, and /dev/stderr · Some commands prefer stdin

- These paths refer to the current standard streams.
- **They** are useful for commands that require filenames.
- They make stream targets explicit in examples.
- Filters often read **stdin** when no files are listed.
- Interactive commands may also read **stdin** for answers.
- Know whether a tool expects filenames or input streams.

Examples

```
# Read current stdin
wc -l /dev/stdin < names.txt
# Write current stderr
echo warning > /dev/stderr
# Filter stdin
sort < names.txt
# Pipe to stdin
printf 'one\ntwo\n' | head -n 1
```

7 Section 7

File Descriptors and exec

7.1 exec can redirect the current shell · exec can capture both streams

- **exec** without a command changes descriptors in the current shell.
- A script can send all later stdout to a log.
- This avoids repeating redirects on every command.
- Redirect stdout first, then point stderr at stdout.
- All following commands share the new destinations.
- This pattern is common at the top of scripts.

Examples

```
# Redirect future stdout
exec > script.log
echo logged
# Redirect future errors
exec 2> script.err
ls missing
exec > run.log 2>&1
echo start
exec >> run.log 2>&1
echo next
```

7.2 Save and restore stdout · fd 3 is a useful extra channel

- Duplicate a descriptor before changing it.
- Restore stdout when only part of a script should be captured.
- Close extra descriptors when done.
- Descriptors 3 and above are available for script use.
- An extra **fd** can keep user messages separate from data.
- Document custom descriptors because they are easy to forget.

Examples

```
# Save stdout
exec 3>&1
exec > part.log
# Restore stdout
exec 1>&3
exec 3>&-
exec 3> debug.log
echo debug >&3
exec 3>&-
```

7.3 Read from an extra descriptor · Append through a descriptor

- A descriptor can be opened for input with <.
- **read** -u reads from a specific descriptor.
- This avoids stealing stdin from another part of a script.
- Opening a descriptor once can simplify repeated writes.
- >> on exec opens the descriptor in **append** mode.
- Writes with >&3 go to that open target.

Examples

```
# Open input fd
exec 3< names.txt
read -r first <&3
# Close input fd
exec 3<&-
# Open append fd
exec 3>> audit.log
echo start >&3
printf 'done\n' >&3
```

7.4 Redirect a loop or block · Descriptor lifetime matters

- A **redirect** after done feeds or captures the whole loop.
- This keeps loop code readable.
- Use it instead of redirecting every command inside.
- A **descriptor** stays open until closed or the process exits.
- Long-lived descriptors can hold files open.
- Close custom descriptors when the job is finished.

Examples

```
# Read loop input
while read -r name; do echo "$name"; done < names.txt
# Capture block output
{ echo one; echo two; } > nums.txt
# Open then close
exec 3> temp.log
exec 3>&-
exec 4< data.txt
exec 4<&-
```

8 Section 8

Temporary Files and Cleanup

8.1 mktemp creates a safe temporary file · mktemp -d creates a temporary directory

- **mktemp** creates a unique path and prints its name.
- It avoids predictable filenames in shared temp directories.
- Store the path in a variable and quote it.
- A temp directory is safer for several related files.
- It gives one cleanup target for a script run.
- Quote the directory path when building filenames.

Examples

```
# Create temp file
tmp=$(mktemp)
printf 'data\n' > "$tmp"
# Show temp path
tmp=$(mktemp)
echo "$tmp"
# Create temp dir
tmpdir=$(mktemp -d)
printf 'x\n' > "$tmpdir/data.txt"
```

8.2 trap cleanup EXIT removes temp files · Keep temp paths private

- An EXIT **trap** runs when the shell exits.
- It cleans up on success and many failure paths.
- Define cleanup after creating the resource.
- Avoid hand-built names like /tmp/app.$$.
- Use mktemp patterns when a suffix or prefix helps.
- Restrict permissions when temp data is sensitive.

Examples

```
# Trap a file
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
# Trap a directory
tmpdir=$(mktemp -d)
trap 'rm -rf "$tmpdir"' EXIT
# Use a template
tmp=$(mktemp /tmp/app.XXXXXX)
tmpdir=$(mktemp -d)
chmod 700 "$tmpdir"
```

8.3 Write temp then move into place · sponge concept for rewriting input

- **Write** complete content to a temporary file first.
- mv within one filesystem is atomic for readers.
- This avoids leaving half-written output files.
- Some pipelines need to read a file before rewriting it.
- **sponge** reads all input before opening the output file.
- Without **sponge**, use a temp file and mv.

Examples

```
# Prepare temp
tmp=$(mktemp)
generate > "$tmp"
# Publish result
mv "$tmp" output.txt
# With sponge
sort names.txt | sponge names.txt
sort names.txt > names.tmp
mv names.tmp names.txt
```

8.4 sync asks the system to flush writes · Checkpoint temporary file habits

- **sync** requests pending filesystem writes be committed.
- **It** is useful before removing media or testing disk writes.
- It does not replace checking command success.
- Create temporary paths with mktemp.
- Register cleanup with trap early.
- Publish final files only after successful writes.

Examples

```
# Flush all writes
sync
# Write then sync
cp image.iso /mnt/usb/
sync
# Minimal pattern
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
cmd > "$tmp" && mv "$tmp" final.txt
```

9 Section 9

Batching and File Streams

9.1 xargs builds commands from stdin · xargs -n controls batch size

- **xargs** reads items and appends them as command arguments.
- **It** is useful when a command does not read stdin itself.
- Check how input is split before using it on real files.
- -n limits how many input items go into each command.
- Small batches are easier to observe and debug.
- Batching can avoid command-line length limits.

Examples

```
# Echo items
printf 'a\nb\n' | xargs echo
# Remove listed files
printf 'old.tmp\n' | xargs rm
# One item each
printf 'a\nb\n' | xargs -n 1 echo
# Two items each
printf 'a\nb\nc\n' | xargs -n 2 echo
```

9.2 Use find -print0 with xargs -0 · xargs -I places items inside commands

- Whitespace and newlines can appear in filenames.
- -print0 separates names with a null byte.
- **xargs** -0 reads those null-delimited names safely.
- -I defines a placeholder token.
- It runs one command per input item by default.
- **Use** it when the item is not the final argument.

Examples

```
# Safe remove
find . -name '*.tmp' -print0 | xargs -0 rm
# Safe count
find . -type f -print0 | xargs -0 wc -l
# Copy by placeholder
printf 'a.txt\n' | xargs -I{} cp {} backup/
# Echo labels
printf 'api\nweb\n' | xargs -I{} echo service={}
```

9.3 Parallel execution needs caution · cat concatenates files

- **Parallel** jobs can speed up independent work.
- They can also overload disks, APIs, or logs.
- Start with small concurrency and make commands idempotent.
- **cat** is ideal for joining files in order.
- It writes file contents to stdout.
- Redirect the combined stream to save it.

Examples

```
# xargs parallelism
printf 'a\nb\n' | xargs -n1 -P2 echo
# GNU parallel style
printf 'a\nb\n' | parallel echo {}
# Show files
cat part1.txt part2.txt
# Combine files
cat part*.txt > combined.txt
```

9.4 split breaks files into pieces · dd copies bytes carefully

- **split** writes chunks with generated suffixes.
- -l splits by line count.
- It helps batch large input for later processing.
- **dd** copies from if= to of= using block sizes.
- **It** is common for device images and byte-limited copies.
- Double-check targets because **dd** can overwrite disks.

Examples

```
# Split by lines
split -l 1000 big.log chunk_
# Split by bytes
split -b 10M archive.bin part_
# Copy first block
dd if=input.bin of=first.bin bs=1M count=1
# Show progress
dd if=image.iso of=/dev/sdX bs=4M status=progress
```

10 Section 10

Filesystem Helpers

10.1 install -m sets mode while copying · install -D creates parent directories

- **install** copies files like cp with extra controls.
- -m sets the destination permissions.
- **It** is useful for scripts and deployment steps.
- -D creates missing destination directories.
- It copies one file into the final path.
- **This** is cleaner than mkdir -p plus cp for one file.

Examples

```
# Install executable
install -m 755 script.sh /usr/local/bin/script
# Install config
install -m 644 app.conf /etc/app.conf
# Create parents
install -D -m 644 app.conf build/etc/app.conf
# Install binary path
install -D -m 755 tool build/bin/tool
```

10.2 ln -s creates symbolic links · readlink -f resolves a path

- A symbolic link points to another path by name.
- **Links** are useful for stable names and version switches.
- Broken links can exist when the target is missing.
- **readlink** -f follows symlinks to a canonical path.
- It also resolves . and .. components.
- Some systems differ, so check portability needs.

Examples

```
# Create link
ln -s releases/v1 current
# Link a command
ln -s /opt/tool/bin/tool ~/bin/tool
# Resolve link
readlink -f current
# Resolve script path
readlink -f "$0"
```

10.3 realpath prints canonical paths · basename extracts the final path component

- **realpath** resolves a path to an absolute form.
- **It** is often clearer than manual cd and pwd tricks.
- Use it when storing paths for later commands.
- **basename** removes leading directory parts.
- It can also strip a suffix.
- Use it when naming outputs from input paths.

Examples

```
# Canonical path
realpath ./data/../data/file.txt
# Store path
root=$(realpath .)
echo "$root"
# Get filename
basename /var/log/app.log
basename report.txt .txt
```

10.4 dirname extracts the parent path · pushd and popd manage directory changes

- **dirname** removes the final path component.
- It returns the directory portion of a path.
- Pair it with mkdir -p before writing outputs.
- **pushd** changes directory and saves the old location.
- popd returns to the previous saved location.
- **They** are useful for scripts that visit several directories.

Examples

```
# Get parent
dirname /var/log/app.log
# Create parent
out=build/logs/app.log
mkdir -p "$(dirname "$out")"
# Enter and return
pushd /tmp
popd
pushd src
make
```

11

Section 11

Disk Awareness and Checkpoint

11.1 dirs shows the directory stack · du -sh summarizes path size

- **dirs** prints locations saved by pushd.
- The leftmost entry is the current directory.
- It helps debug nested directory changes.
- **du** reports disk usage for files and directories.
- -s summarizes each argument.
- -h prints human-friendly units.

Examples

```
# Show stack
dirs
# Clear stack
dirs -c
# Directory size
du -sh logs/
# Several paths
du -sh build dist cache
```

11.2 df -h shows filesystem free space · pv previews pipeline progress

- **df** reports space at the filesystem level.
- -h prints sizes in human-friendly units.
- Check **df** when writes fail despite small files.
- **pv** can show throughput and progress for stream data.
- **It** is optional and may not be installed everywhere.
- Use it between producer and consumer commands.

Examples

```
# All filesystems
df -h
# One path
df -h .
# Show copy progress
pv image.iso > /tmp/image.iso
# Progress in pipe
tar cf - dir | pv | gzip > dir.tar.gz
```

11.3 Compare common redirect mistakes · Choose log locations deliberately

- > replaces while >> appends.
- 2>&1 depends on the stdout target at that point.
- A pipe carries stdout only unless stderr is merged.
- Logs should go somewhere predictable.
- Create log directories before redirecting into them.
- Use absolute paths for long-running scheduled jobs.

Examples

```
# Append instead of replace
echo next >> app.log
# Merge before pipe
cmd 2>&1 | tee all.log
# Create log dir
mkdir -p logs
./job.sh >> logs/job.log 2>&1
log="$HOME/logs/job.log"
./job.sh >> "$log" 2>&1
```

11.4 Review files and pipes · Module 05 checkpoint

- Use streams to connect commands cleanly.
- Choose redirects based on which descriptor should move.
- Protect important files with temp paths and careful appends.
- Explain where stdout, stdin, and stderr are going.
- Build a pipeline that saves and displays output.
- Clean up temporary files even when a script exits early.

Examples

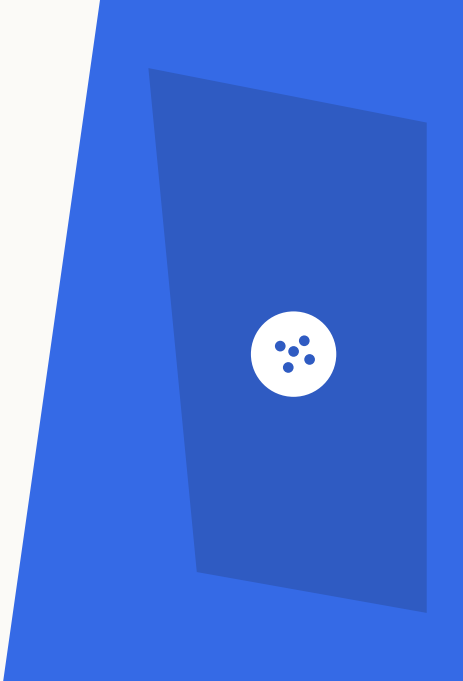
```
# Clean pipeline
grep ERROR app.log | sort | uniq -c
# Safe capture
./job.sh > job.out 2> job.err
# Checkpoint pipeline
find . -type f -print0 | xargs -0 wc -l
tmp=$(mktemp)
trap 'rm -f "$tmp"' EXIT
```

NEXT STEP

Practice every example in your terminal

Then open the next module deck

Merit Advisory
05 · Files & Pipes



July 2026